

Solidity

27/09/25

1st lesson : Hello world



This is a beginner-friendly cheat sheet, made by a beginner for beginners.

⚠ I'm still learning, so there may be small mistakes — don't hesitate to double-check!

Note: Keywords in **bold white** have their own mini flashcards at the end.

1. Key concepts

- **Solidity**: a programming language for writing smart contracts on Ethereum.
- **Smart contract**: a program that runs automatically on the blockchain. It stores data and defines rules or actions that execute when called. Once deployed, its code and rules are immutable and transparent for everyone to see.

2. Basic Contract Structure

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract MyContract {
5     // variables
6     // functions
7 }
```

• **SPDX-License-Identifier** specifies **the license** (MIT here).

• **pragma solidity** defines the **compiler version**.

• **contract [MyContract] { ... }** defines the contract.

3. Hello World example's explanation

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity >=0.7.0 <0.9.0;
3 contract HelloWorld
4 {
5     string private stateVariable = "Hello World";
6     function GetHelloWorld() public view returns ( string memory)
7     {
8         return stateVariable;
9     }
10
11 }
```

Create a state variable structure

```
type visibility StateVariableName = "value";
```

- **Type**: string
- **Visibility**: private (only accessible via a function)
- State Variable Name : stateVariable
- "value" : Stores the message

Aims to store the message on the blockchain, but only accessible and manageable within the contract itself (private).

Private : Stores securely on the blockchain and prevents anyone from changing it directly.

Public : Lets anyone read the value of "Hello World" safely without giving direct access to the variable itself.

Create a function

```
function FunctionName () visibility modifier returns (type memory)
{
    return StateVariableName;
}
```

- **function** : mandatory. To declare the function
- fonction name : here GetHelloWorld
- () : **parameters** (empty here).
- visibility : public (who can call).
- **modifier** : the rule (view = any modification)
- returns (type memory) : output
- return stateVariable : gives the value Hello World

The function reads the state variable and returns its value; it is public, so anyone can call it.

Licences

- Determines what others can do with your code.
- No license = no one can legally reuse your code.
- With a license, you can allow copying, modifying, sharing.

Solidity & smart contracts

Always add at the top of your file:

```
// SPDX-License-Identifier: MIT
```

Applies to the code, not to any tokens created by the contract.

Example of licences

License	What you can do	Main condition
MIT	Copy, modify, share, even commercially	Keep copyright and license notice
Apache 2.0	Same as MIT + patent protection	Include copyright + patent clause
GPL	Copy, modify, share	Derived projects must also be GPL
BSD	Same as MIT	Keep copyright notice
Unlicense / CC0	Fully free	No conditions, free use

For a beginner or educational project, MIT is perfect.

Types

What is a type in Solidity

A type defines the kind of data a variable or function can hold or return.

Type	Description	Example
string	text / character string	"Hello"
int	Signed integer	-5
uint	positive integer	25
bool	true / false	true
address	Ethereum address	0x123456789...
bytes	binary data	0x5f16c8f58...

Compiler version

What is a compiler?

A compiler is a program that translates your Solidity code into bytecode that the Ethereum Virtual Machine (EVM) can execute.



Accepts versions from 0.7.0 up to, but not including, 0.9.0.

```
pragma solidity >=0.7.0 <0.9.0;
```



^0.8.0 → compatible with version 0.8.0 and all minor versions above it (0.8.x).

```
pragma solidity ^0.8.0;
```

Solidity evolves quickly with new features, bug fixes, and changes in syntax. That's why specifying the compiler version is important (and necessary).



[Sopiloo](#) 27/09/25

State Variable

What is a state variable in Solidity ?

- A state variable is **a value stored permanently on the blockchain**. It represents the data or state of a smart contract.
- Its value can be read or modified by the contract's **functions**, depending on its **visibility**.

Why is it useful?

- Creates a permanent, shared state.
- Allows functions to read or modify it to execute logical rules.
- Stores reliable, permanent data that can be used for automatic and verifiable actions.

Visibility

Visibility defines who can access a variable or function in a contract.

visibility	Access
public	Everyone + contract itself
private	Only inside the contract
internal	Contract + inherited contracts
external	Only from outside the contract

- Controls access and security of contract data.
- Helps protect sensitive variables or functions from unauthorized use.

Function

- A function is a block of code in a smart contract that **performs an action**.
- It can read or modify state variables and return values.
- It may have modifiers like public, private, view, or pure to control access and behavior.

Parameters

- Parameters are inputs that you give to a function when calling it.
- They allow the function to perform actions based on the values provided.

Purpose:

- Make functions flexible and reusable with different data.
- Control how the function modifies state variables or computes results.

Modifier

- A modifier is **a rule or condition** applied to a function in a smart contract.
- It controls who can call the function or under what conditions it executes.

Example of modifier :

- **view** : ensures the function only reads data and does not modify the blockchain.
- **pure** : ensures the function neither reads nor writes to the blockchain; it only does calculations.
- **payable** : allows the function to receive Ether when called.

Custom modifiers : you can define any rules, like checking a user's balance, verifying a condition, or restricting access to certain roles.